Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Greedy Algorithms

Prof. Reza Skandari

Today

1. **Greedy algorithms and the knapsack problem**
2. Wrap-up

Greedy algorithms

A "class" of algorithms, or way of thinking

- Others: dynamic programming, divide and conquer...

Make simple **myopic** decisions iteratively

Greedy algorithms are:

- Easy to come up with...
- ...but **not** always correct

Knapsack problem

Problem: fill a bag with the most valuable items available.

Input:

- Set of n items, with values v_i and sizes w_i (integer)
- Capacity W

Output: subset S of items that maximizes the sum of values subject to a capacity constraint:

- maximize $\sum_{i \in S} v_i$
- subject to $\sum_{i \in S} w_i \leq W$

Knapsack problem applications





Many problems with budget constraints are versions of knapsack

- Selecting portfolios (e.g. projects to invest in)
- Lots of "operational" problems...

Choosing a subset

Items to pack

- **Shirt**: value 5, weight 5
- **Bottle**: value 10, weight 5

Subsets: (size W limits **feasible** subsets)

- {}, {Shirt}, {Bottle}, {Shirt, Bottle}

Exhaustive (brute-force) search through all possibilities?

- Go through all subsets of $\{1, 2, 3, ..., n\}$
- Suppose we have 50 items: $O(2^n)$ subsets...

A simpler approach?

Input: n items with values v_i, sizes w_i; capacity W

Output: subset S maximizing $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$

Greedy solution? — Pick myopically without worrying about future choices

- **Highest-value** item first?

Running time

Greedy algorithm:

- Sort items by value
- Pick items until capacity full

Running time?

- Sorting n items
- Loop through the sorted n items to fill knapsack
- Total $O(n \log n)$

Does it work? Attempt 1: highest value first

Example: capacity $W = 20$, three items with values $v = [10, 10, 11]$, weights $w = [10, 10, 11]$.

- Picking highest value first is **bad**...
  - We take the item worth 11 (weight 11), leaving only 9 capacity — nothing else fits
  - But the two items worth 10 each (weight 10 + 10 = 20) fit exactly, for a total of 20 > 11

Does it work? Attempt 2: lowest weight first

Example: capacity $W = 20$, three items with values $v = [10, 10, 50]$, weights $w = [10, 10, 11]$.

- Picking lowest weight first is **bad**...
  - We take both weight-10 items (value 20), leaving no room for the weight-11 item worth 50
  - Taking just the single weight-11 item would have given 50 > 20

Does it work? Attempt 3: value per weight ("bang for buck")

What about combining value and size?

- **Sort items by value per weight** v_i/w_i, and pick in this order

Example: capacity $W = 510$, three items with values $v = [10, 10, 500]$, weights $w = [10, 10, 501]$.

- Greedy picking is **bad**...
  - The two small items have the best ratio ($10/10 = 1$ each) and get picked first, filling 20 of the 510 capacity
  - But there's no room left for the item worth 500 (weight 501 > 490 remaining)

So does anything work?

None of the three greedy heuristics is guaranteed correct

It would be optimal if we could **divide items into fractions** — but our items are all-or-nothing (binary)

For the binary problem, guaranteeing the correct solution needs a different approach: **dynamic programming**

- Exploit problem structure to loop through all items and possible capacities
- Correct solution, $O(nW)$ running time ("pseudo-polynomial")

Improving the greedy heuristic

Greedy knapsack (bang-for-buck) was sometimes **incorrect**, but very **fast**: $O(n \log n)$

How bad is it? Worst case: leave out a single extremely valuable item that would have fit

Modified algorithm: run two greedy heuristics, keep the better result

- Sort items in decreasing v_i/w_i, pick until full
- *Also* sort items in decreasing v_i, pick until full
- Take whichever of the two results is better
- Runtime complexity unchanged: still $O(n \log n)$
- Removes the single-item worst-case scenario
- In practice, items are often small relative to W — greedy choices rarely leave much value out

Why not just always use dynamic programming?

DP gives the **exact** answer for knapsack in $O(nW)$ time

But W can be a huge number — and $O(nW)$ time depends on the *value* of W, not just how many digits it takes to write down

This turns out to matter a lot. Let's see why.

Polynomial vs. pseudo-polynomial time?

Polynomial running time: $O(n^k)$ for some k. A problem is:

- **Tractable** = solvable in polynomial time for any input
- **Intractable** = NOT solvable in polynomial time

Pseudopolynomial time = "polynomial for us, exponential for a computer"

- W is a large integer, e.g. $W = 2329316...8754$
- Input size for a computer is the number of binary digits (bits) in W
- In binary, W has $x = \log_2(W)$ digits — "how many times can we divide W by 2 to get to 1"
- So $W = 2^x$: **exponential** in the input size the computer actually sees

Many important problems are intractable

Tractable problem = solvable in polynomial time $O(n^k)$ for some k

Finding the **shortest route to a Pokémon nest** (Dijkstra's algorithm) vs. finding the **shortest tour of all Pokémon nests**



Example of intractability: **travelling salesperson problem** (TSP)

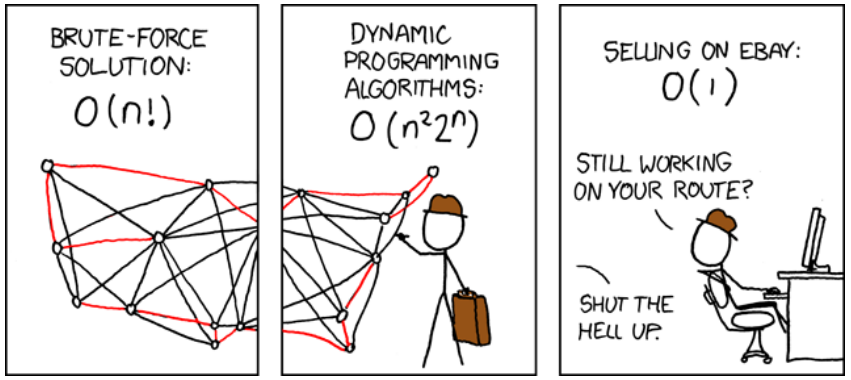
- **Input**: undirected graph with non-negative edge costs
- **Goal**: find minimum-cost tour visiting every node
- **Conjecture**: no polynomial-time algorithm gives the correct solution — find an approximate solution instead

Recent advances in TSP

"... [U of Washington researchers] were able to subtract 0.2 billionth of a trillionth of a trillionth of a percent [out of Christofides' 50% guarantee]."

"This is a result I have wanted all my career," said David Williamson of Cornell University, who has been studying the travelling salesperson problem since the 1980s.

Quanta Magazine, October 2020.



Review

Arc 1 — Knapsack: greedy heuristics are fast ($O(n \log n)$) but not guaranteed correct; dynamic programming is correct but only pseudo-polynomial ($O(nW)$)

Arc 2 — Why this matters more broadly: some problems (like TSP) aren't just slow to solve exactly — no efficient exact algorithm is known at all. Greedy algorithms, approximations, and heuristics are often the only practical option

Exercises: knapsack and fantasy football

Today

1. Greedy algorithms and the knapsack problem
2. **Wrap-up**

Parting words

Module wrap-up

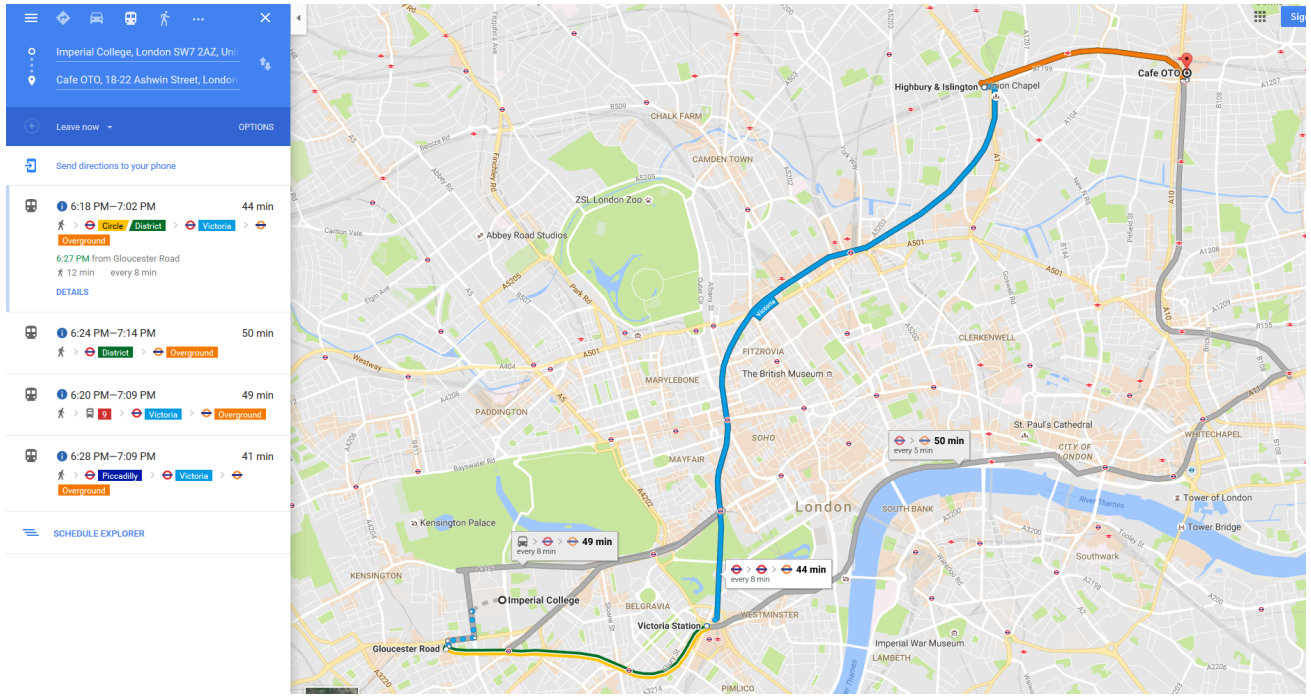
Abstraction

Variables, functions, objects

```
=MIN($D$67-SUM(H$73:H75),MAX(IF($F$51="yes",MAX(0,MIN(
$F76,$D$67-SUM($F$73:F75))))+IF(F76=0,($D$67-SUM(H$73:
H75))*C76/(SUM(C76:C$80),0))+MIN(MAX(0,($F$52/$F$50*
$F76)-MAX(0,MIN($F76,$D$67-SUM($F$73:F75))))+MAX(0,IF
(F76=0,($D$67-SUM(H$73:H75))*C76/(SUM(C76:C$80)),0)),0),
0))+MAX(0,IF(F76=0,($D$67-SUM(H$73:H75))*C76/(SUM(C76:
C$80)),0))+MAX(0,IF(F76=0,($D$67-SUM(H$73:H75))*C76/
(SUM(C76:C$80)),0)))
```

Source: Quora

Algorithms



Source: Google Maps

Recipes for solving problems

Programming languages

Python has brought computer programming to a vast new audience

And its inventor has just stepped down



Over to you

Turn to the person next to you (2 minutes):

- What's one thing you built or debugged this term that you're actually proud of?
- What's a concept that finally *clicked* — and when did it click?

A few of you will share with the room.

What next?

What are you excited to try now that you can code?

A personal project? A work idea? The MSc? Something else entirely?

Since you asked: what we did *not* do

- **Analytics** (!)
- **Write Python → ??? → Result**
  - How code actually runs: read *"Code"* by Charles Petzold
- **Mathematical proofs**
- **Many more interesting algorithms and data structures...**
  - *"Introduction to Algorithms"* by Cormen et al.

What next?

- The MSc
- Your own projects